

数据	组织	
	移动	i 将 i 设置为 9
循环	调试 (十六进制长 (i))	'debug, output i in hex
	条件	i,#loop 如果 i 递减且不为 -1, 则循环。
	跳转	跳转 # \$ 别乱跑, 待在这里。
i	-	预留一个注册表作为 'i'

运行 Ctrl-F10 后, 会打开调试窗口, 如下所示:

```
Cog0 初始化 $0000_0000 $0000_0000 加载
Cog0 i -
Cog0 i -
Cog0 i -
Cog0 i -
Cog0 i -
Cog0 i -
Cog0 i - 4
Cog0 i -
Cog0 i -
Cog0 i -
Cog0 i -
Cog0 i -
```

在第一个标题 报告中的一行, 您会看到 Cog0 从 \$00000 加载我们的 PASM 程序。之后, 程序运行, 您会看到 'i' 从 9 递减迭代。0.

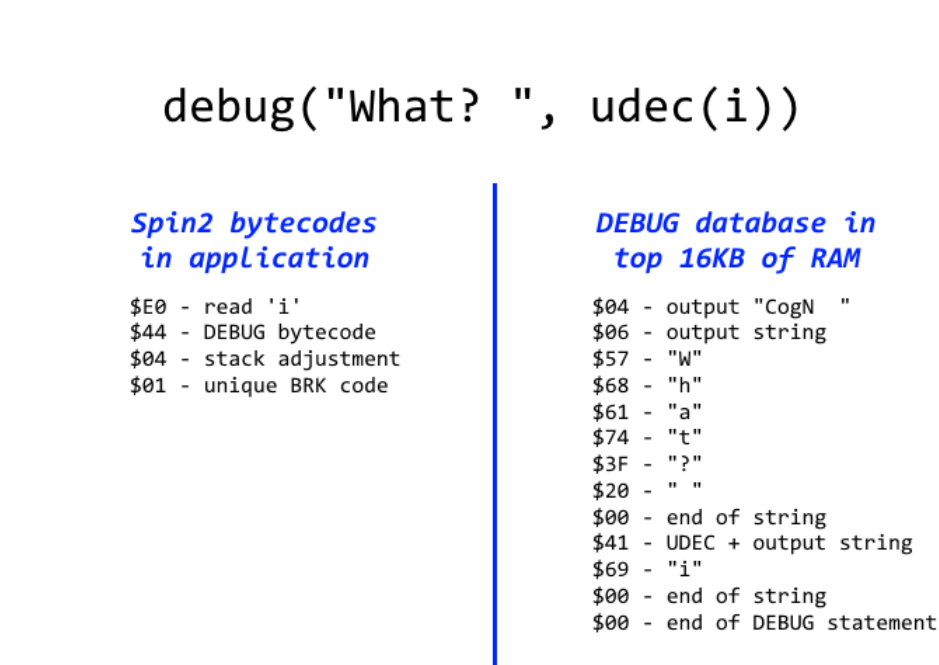
“调试内存使用”中
如果在 MOV 指令中将 “9” 改为 “99” 并希望减慢速度, 请在 DEBUG 语句中添加一个 DLY 指令, 并确保将毫秒数表示为 #250, 因为一个普通的 250 会被解释为寄存器 250:

调试 (十六进制长 (i), dly(#250)) ' 调试, 报告后输出 i 的十六进制值并延迟 250 毫秒

调试 内存利用率

Here is what the memory utilization looks like for 一个 Spin2 调试语句。您可以在 Spin2 侧看到, 读取变量 'i' 需要字节码, 然后是三个强制性的 bytecodes make up the actual DEBUG instruction.

栈调整字节指示解释器将栈向下移动多远, 以便有效地 “弹出” 为调试事件准备的所有表达式。在这种 “i” 的情况下, 栈只需要向下移动四个字节 (一个长)。当调用调试器时, 调试器所需的值将排列在当前的 Spin2 栈指针正上方。



The 'unique BRK code' byte is used as an ind查找 DEBUG 数据库中位于顶部的特定记录。emory, from which the debugger reads its commands.

In the case where DEBUG is active, but a cog has had its debug interrupt disabled via the DEBUG_ENABLE symbol, Spin2 DEBUG instructions will not trigger a debug interrupt, but they do still pop any DEBUG-intended values from the stack, so these are harmless events.

对于 PASM DEBUG 语句, 在 DEBUG 命令插入的位置插入一个 'BRK #code' 指令, 所有相关数据都存储在 DEBUG 数据库中。如果齿轮的 debug 中断已禁用, 'BRK #code' 指令将不起作用, 消耗两个时钟。

DAT	ORG	
	MOV	i,#9 'set i to 9
loop	DEBUG	(UHEX_LONG(i)) 'debug, output i in hex
	DJNF	i,#loop 'decrement i and loop if not -1
	JMP	# \$ 'don't go wandering off, stay here
i	RES	1 'reserve one register as 'i'

When run with Ctrl-F10, the Debug window opens and this is what appears:

```
Cog0 INIT $0000_0000 $0000_0000 load
Cog0 i = $0000_0009
Cog0 i = $0000_0008
Cog0 i = $0000_0007
Cog0 i = $0000_0006
Cog0 i = $0000_0005
Cog0 i = $0000_0004
Cog0 i = $0000_0003
Cog0 i = $0000_0002
Cog0 i = $0000_0001
Cog0 i = $0000_0000
```

In the first line of the report, you see Cog0 loading our PASM program from \$00000. After that, the program runs and you see 'i' iterating from 9 down to 0.

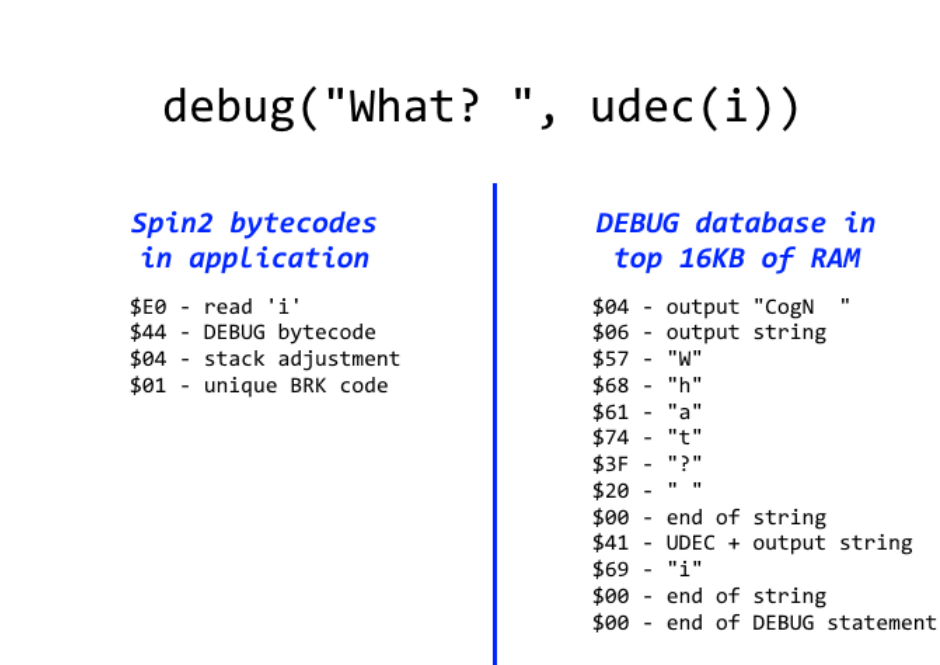
If you change the "9" to "99" in the MOV instruction and you'd like to slow things down, add a DLY command to the DEBUG statement and be sure to express the milliseconds as #250, since a plain 250 would be understood as register 250:

debug (uhex_long(i), dly(#250)) 'debug, output i in hex and delay for 250ms after each report

DEBUG memory utilization

Here is what the memory utilization looks like for a Spin2 DEBUG statement. You can see, on the Spin2 side, that a bytecode is needed to read the variable 'i', and then three obligatory bytecodes make up the actual DEBUG instruction.

The 'stack adjustment' byte tells the interpreter how far to drop the stack to effectively 'pop' all the expressions that were pushed in preparation for the DEBUG event. In this case of 'i', only, the stack needs to drop by four bytes (one long). When the debugger is invoked, the values it needs will be ordered right above the current Spin2 stack pointer.



The 'unique BRK code' byte is used as an index to look up the specific record in the DEBUG database at the top of memory, from which the debugger reads its commands.

In the case where DEBUG is active, but a cog has had its debug interrupt disabled via the DEBUG_ENABLE symbol, Spin2 DEBUG instructions will not trigger a debug interrupt, but they do still pop any DEBUG-intended values from the stack, so these are harmless events.

For PASM DEBUG statements, a 'BRK #code' instruction is inserted where the DEBUG command was placed, and all related data resides in the DEBUG database. If the cog's debug interrupt is disabled, the 'BRK #code' instruction does nothing, taking two clocks.

调试和中断

在调试语句期间接收到的中断请求将在调试完成后执行，但响应时间可能严重失调，导致中断的重新触发设置无法正确进行。高频周期性智能引脚中断尤其容易出现此问题。假设你在正常的 ISR（中断服务例程）中执行一个 AKPIN 指令，以降低 INA/INB 信号，使智能引脚再次升高，从而触发一个新的中断。与此同时，在 AKPIN 和 RETIx 之间，智能引脚触发，使 INA/INB 升高。这仅仅是因为你的时序周期由于调试语句而失调。由于 ISR 忙碌时发生的中断不会被看到，这会导致中断停止循环。然而，CT 中断不太容易出现此问题，因为它们有 8000_0000 个时钟周期来被识别。为了解决智能引脚重新触发问题，你可以触发 INA/IIN 高，而不是 INA/IIN 上升，但这可能会导致你的智能引脚配置出现性能问题。

解决此调试 / 中断困境的一种安全方法是，只对后台未执行中断服务例程的齿轮执行调试语句。如果中断服务例程能够容忍时间偏差，并且没有挂起中断循环的风险，则可以在了解中断时间退化的情况下执行调试语句。

图形化调试显示

调试消息可以调用内置到工具中的特殊图形化调试显示。这些图形化显示每个都以一个独特的窗口形式呈现。一旦实例化，显示就可以持续接收数据以生成动态可视化。这些显示对于开发和调试非常方便，因为各种数据类型可以在其原生上下文中查看。最多可以同时运行 32 个图形化显示。

当调试消息包含反引号 (') 字符 (ASCII \$60) 时，包含从反引号到消息末尾的所有内容的字符串，将被发送到图形调试显示解析器。解析器查找多种不同的元素类型，并将所有逗号视为空白：

元素类型	DEBUG memory utilization	Graphical DEBUG Displays	描述
显示类型	逻辑，示波器，绘图，位图。		这是您想要实例化 Graphical 调试显示类型的正式名称。
未知符号	MyLogicDisplay		每个图形调试显示实例必须指定一个唯一的符号名称。
instance_name	MyLogicDisplay		实例化后，一个图形化的调试显示实例通过其符号名称进行引用。
keyword	标题，POS，大小，样本		关键字用于配置显示。它们后可能跟数字、字符串和其他关键字。
number	1024, \$FF, %1010		数字可以用十进制、十六进制（\$）和二进制（%）表示。
字符串	这是一个字符串		字符串用单引号表示。

在我们深入了解所有这些如何配合使用之前，我们需要回顾一些可以用于显示器的特殊调试显示语法。当调试语句中的第一个字符是反引号时，就会调用此语法。这会导致调试语句中的所有内容都视为字符串，除非后跟的反引号充当“转义”字符，以允许正常的或缩短的调试命令。

调试语句 (v=100, BYTE[a]=1,2,3,4,5)	调试 Message 输出	注意
调试 (" ` 逻辑 MyDisplay 样本 ", SDEC_(v))	逻辑 MyDisplay 样本 100	常规调试语法可以驱动调试显示，但并非最佳。
调试 (LOGIC MyDisplay 样本 10)	逻辑 MyDisplay 样本 100	调试 - 显示语法更简单，输出中省略了 'CogN'。
调试 (LOGIC MyDisplay 样本 (v))	逻辑 MyDisplay 样本 100	十进制数使用 `(value) 符号输出。是 SDEC_ 的缩写。
调试 (LOGIC MyDisplay 样本 i)	逻辑 MyDisplay 样本 \$64	十六进制数使用 `\$(value) 符号输出。是 UHEX_ 的缩写。
调试 (LOGIC MyDisplay 样本 %(v))	LOGIC MyDisplay 样本 <10> 左对齐 </10>	二进制数使用 `% (value) 符号输出。UBIN_ 的缩写。
调试 ` 逻辑 MyDisplay 标题 ` `#(v)'	逻辑 MyDisplay 标题 'd'	字符使用 `#(value) 符号输出。
调试 (MyDisplay UDEC_BYTE_ARRAY_(@a,5))	MyDisplay 1, 2, 3, 4, 5	常规调试命令也可以后跟反引号。

使用图形化调试显示有两步：第一步，它们必须被实例化；第二步，它们必须被输入：

使用显示：	1st	第二	第三	第四	注意
首先，实例化它。	`	显示类型	未知符号	关键词 (s), 数字 (s), 字符串 (s)	未知符号 变为 instance_name。
调试内存使用图形调试显示	`	实例名称 (s)	关键词 (s), 数字 (s), 字符串 (s)		多个显示器可以接收相同的数据。

为了将所有内容结合起来，让我们在示波器上显示一个锯齿波。 lay:

DEBUG and interrupts

Interrupt requests received during a DEBUG statement will execute after the DEBUG completes, but the response time may be so skewed that the retrigger setup for the interrupt won't happen properly. High-frequency cyclical smart pin interrupts are especially prone to this problem. Imagine you do an AKPIN instruction within your normal ISR (interrupt service routine) to drop the INA/INB signal so that the smart pin can make it go high again, triggering a new interrupt. Meanwhile, after the AKPIN and before the RETIx, the smart pin triggers, raising INA/INB high. This is only happening because your cycle-frame timing has become skewed from the DEBUG statement. This interrupt won't be seen since it happened when the ISR was busy. This will cause the interrupt to cease cycling. CT interrupts are not prone to this problem, though, since they have \$8000_0000 clock cycles in which to be recognized. To remedy the smart-pin retrigger problem, you could trigger on INA/INB-high, as opposed to INA/INB-rise, but this could cause performance problems with your smart pin configurations.

One fail-safe way to get around this DEBUG/interrupt dilemma is to only do DEBUG statements from cogs that are not executing ISRs in the background. If the ISRs can tolerate timing skew and there is no risk of hanging interrupt cycling, you can do DEBUG statements with some understood interrupt timing degradations.

Graphical DEBUG Displays

DEBUG messages can invoke special graphical DEBUG displays which are built into the tool. These graphical displays each take the form of a unique window. Once instantiated, displays can be continuously fed data to generate animated visualizations. These displays are very handy for development and debugging, as various data types can be viewed in their native contexts. Up to 32 graphical displays can be running simultaneously.

When a DEBUG message contains a backtick (') character (ASCII \$60), a string, containing everything from the backtick to the end of the message, is sent to the graphical DEBUG display parser. The parser looks for several different element types, treating any commas as whitespace:

Element Type	Example	Description
display_type	LOGIC, SCOPE, PLOT, BITMAP	This is the formal name of the graphical DEBUG display type you wish to instantiate.
unknown_symbol	MyLogicDisplay	Each graphical DEBUG display Instance must be given a unique symbolic name.
instance_name	MyLogicDisplay	Once instantiated, a graphical DEBUG display instance is referenced by its symbolic name.
keyword	TITLE, POS, SIZE, SAMPLES	Keywords are used to configure displays. They might be followed by numbers, strings, and other keywords.
number	1024, \$FF, %1010	Numbers can be expressed in decimal, hex (\$), and binary (%).
string	'Here is a string'	Strings are expressed within single-quotes.

Before getting into how all this fits together, we need to go over some special DEBUG-display syntax that can be used for displays. This syntax is invoked when the first character in the DEBUG statement is the backtick. This causes everything in the DEBUG statement to be viewed as a string, except when subsequent backticks act as 'escape' characters to allow normal or shorthand DEBUG commands.

DEBUG Statement (v=100, BYTE[a]=1,2,3,4,5)	DEBUG Message Output	Note
DEBUG(" `LOGIC MyDisplay SAMPLES ", SDEC_(v))	Cog0 `LOGIC MyDisplay SAMPLES 100	Regular DEBUG syntax can drive DEBUG displays, but it's not optimal.
DEBUG(`LOGIC MyDisplay SAMPLES 100)	`LOGIC MyDisplay SAMPLES 100	DEBUG-display syntax is simpler and 'CogN' is omitted in the output.
DEBUG(`LOGIC MyDisplay SAMPLES `(v))	`LOGIC MyDisplay SAMPLES 100	Decimal numbers are output using `(value) notation. Short for SDEC_.
DEBUG(`LOGIC MyDisplay SAMPLES `\$(v))	`LOGIC MyDisplay SAMPLES \$64	Hex numbers are output using `\$(value) notation. Short for UHEX_.
DEBUG(`LOGIC MyDisplay SAMPLES `%(v))	`LOGIC MyDisplay SAMPLES %1100100	Binary numbers are output using `% (value) notation. Short for UBIN_.
DEBUG(`LOGIC MyDisplay TITLE ` `#(v)')	`LOGIC MyDisplay TITLE 'd'	Characters are output using `#(value) notation.
DEBUG(`MyDisplay `UDEC_BYTE_ARRAY_(@a,5))	`MyDisplay 1, 2, 3, 4, 5	Regular DEBUG commands can follow the backtick, as well.

There are two steps to using graphical DEBUG displays. First, they must be instantiated and, second, they must be fed:

To Use a Display:	1st	2nd	3rd	4th	Note
First, instantiate it.	`	display_type	unknown_symbol	keyword(s), number(s), string(s)	Unknown_symbol becomes instance_name.
Then, feed it.	`	instance_name(s)	keyword(s), number(s), string(s)		Multiple displays can be fed the same data.

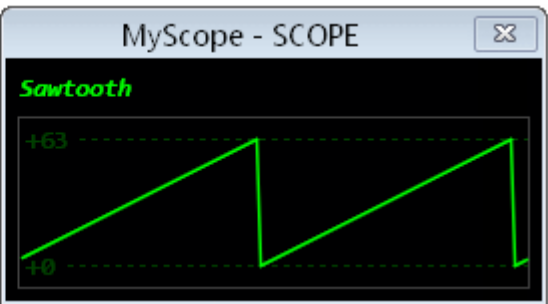
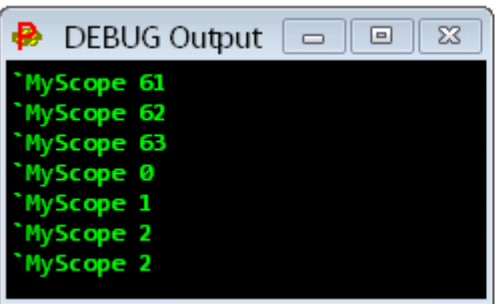
To bring this all together, let's show a sawtooth wave on a SCOPE display:

```
CON _clkfreq = 10_000_000

PUB go() | i

    debug(`SCOPE MyScope SIZE 254 84 SAMPLES 128)
    debug(`MyScope 'Sawtooth' 0 63 64 10 %1111)

    repeat
        debug(`MyScope `(i & 63))
        i++
        waitms(50)
```



在上面的示例中，有一个名为 MyScope 的 SCOPE 实例，其尺寸为 254 x 84 像素，并显示了 128 个样本。选择宽度为 254，因为样本编号为 0..127，我希望它们以恒定的两像素间距（127 * 2 = 254）分布。选择高度为 84，以便在波形上方和下方各有 10 个像素，波形的高度为 64 像素。定义了一个名为“锯齿波”的通道，其定义目的为显示，底部值为 0，顶部值为 63，在指定范围内有 64 像素高，并且在范围的底部向上突出 10 个像素。%11 启用顶部和底部图例值以及顶部和底部线。在 REPEAT 块中，SCOPE 接收一个重复的 0.63 模式，该模式形成锯齿波。SCOPE 每次接收到值时都会更新其显示。如果定义了八个通道而不是一个，它将在每接收到第八个值时更新显示，绘制所有八个通道。

Currently, the following g图形调试显示已实现，但将添加更多。e future:

显示类型	描述
逻辑	带有单位和多位标签的逻辑分析仪，可触发 1..32 个通道上的模式。
示波器	带有 1..8 个通道的示波器，可以在具有滞后效应的电平上触发。
示波器_XY	带有 1..8 个通道的 XY 示波器，持续时间为 0..512 个样本，极坐标模式，对数刻度模式
快速傅里叶变换	使用 1..8 个通道、 4..2048 个点的快速傅里叶变换，窗口化结果，对数刻度模式
频谱	包含 4..2048 点 FFT、窗口化结果、相位着色和对数刻度模式的光谱仪
绘图	通用绘图仪，支持笛卡尔和极坐标模式
终端	文本终端，大小为 300 x 200 字符，字号为 6-200 点，支持 4 种同时颜色方案。
位图	位图，1..2048 x 1..2048 像素，1/2/4/8/16/32 位图，包含 19 种颜色系统，15 种方向 / 自动滚屏模式，独立 X 和 Y 像素大小为 1..256
MIDI	带 1..128 键、力度显示、可变屏幕比例的钢琴键盘

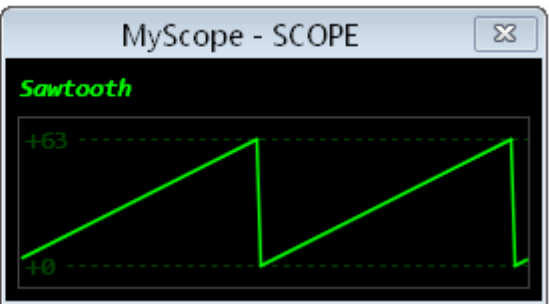
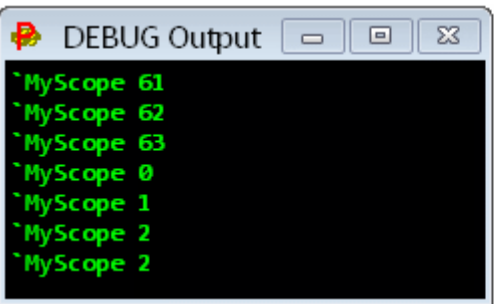
Following are elaborations of each DEBUG display type.

```
CON _clkfreq = 10_000_000

PUB go() | i

    debug(`SCOPE MyScope SIZE 254 84 SAMPLES 128)
    debug(`MyScope 'Sawtooth' 0 63 64 10 %1111)

    repeat
        debug(`MyScope `(i & 63))
        i++
        waitms(50)
```



In the example above, a SCOPE is instantiated called MyScope that is 254 x 84 pixels and shows 128 samples. A width of 254 was chosen since samples are numbered 0..127 and I wanted them to be spaced at a constant two-pixel pitch (127 * 2 = 254). A height of 84 was chosen so that there would be 10 pixels above and below the waveform, which will have a height of 64 pixels. A channel called "Sawtooth" is defined which, for the purpose of display, has a bottom value of 0 and a top value of 63, is 64 pixels tall within that range, and is elevated 10 pixels off the bottom of the scope window. The %1111 enables top and bottom legend values and top and bottom lines. Within the REPEAT block, the SCOPE is fed a repeating pattern of 0..63 which forms the sawtooth wave. The SCOPE updates its display each time it receives a value. If there were eight channels defined, instead of just one, it would update the display on every eighth value received, drawing all eight channels.

Currently, the following graphical DEBUG displays are implemented, but more will be added in the future:

Display Types	Descriptions
LOGIC	Logic analyzer with single and multi-bit labels, 1..32 channels, can trigger on pattern
SCOPE	Oscilloscope with 1..8 channels, can trigger on level with hysteresis
SCOPE_XY	XY oscilloscope with 1..8 channels, persistence of 0..512 samples, polar mode, log scale mode
FFT	Fast Fourier Transform with 1..8 channels, 4..2048 points, windowed results, log scale mode
SPECTRO	Spectrograph with 4..2048-point FFT, windowed results, phase-coloring, and log scale mode
PLOT	General-purpose plotter with cartesian and polar modes
TERM	Text terminal with up to 300 x 200 characters, 6..200 point font size, 4 simultaneous color schemes
BITMAP	Bitmap, 1..2048 x 1..2048 pixels, 1/2/4/8/16/32-bit pixels with 19 color systems, 15 direction/autoscroll modes, independent X and Y pixel size of 1..256
MIDI	Piano keyboard with 1..128 keys, velocity depiction, variable screen scale

Following are elaborations of each DEBUG display type.

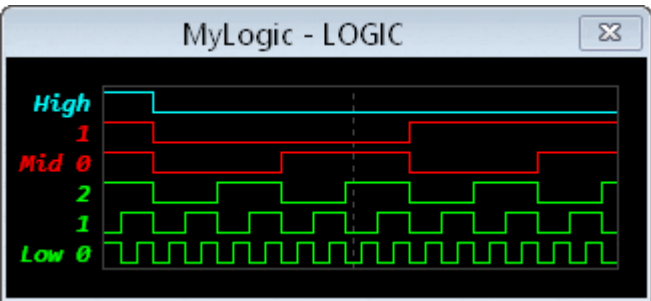
LOGIC 显示逻辑分析仪，具有单位和多位标签，1..32 通道，可以按模式触发。

```
CON_clkfreq 1_000_000

PUB go() | i

    调试` 逻辑 MyLogic 样本 32 '低' 3 '中' 2 '高'
    调试` MyLogic 触发 $07 $04 休止时间 2

    重复
        调试 MyLogic i & 63
    _
        等待 (25)
```



逻辑 实例化	描述	默认
标题 'string'	将窗口标题设置为“字符串”。	无
POS 左上角	设置窗口位置。	0, 0
样本 4_to_2048	设置要跟踪和显示的样本数。	32
间距 2_to_32	设置样本间距。显示器的宽度为 SAMPLES * 间距。	8
速率 1_to_2048	在每次显示更新之前，设置样本数（或触发数，如果已启用）。	1
线条大小 1_到_7	设置线条粗细。	1
文本大小 6_to_200	设置图例文本大小。文本高度决定逻辑电平的高度。	编辑器文本大小
颜色 背景颜色 网格颜色	设置背景和网格颜色。	黑色，灰色 4
名称 {1_to_32 { 颜色 }}	设置第一个 / 下一个通道或组名称，可选位计数，可选颜色 *。	默认颜色
打包数据模式	启用打包数据模式。请参阅本节末尾的描述。	无
逻辑 喂养	描述	默认
触发 掩码匹配 样本偏移量	当（数据 & 掩码）匹配时触发 = 0。如果掩码 = 0，则触发被禁用。	0, 1, 样本 / 2
休止时间 2 到 2048	设置从触发到触发所需的最小样本数。	样本
数据	数值数据以 LSB 优先的方式应用于通道。	
清除	清除采样缓冲区和显示，等待新数据。	
保存 { 窗口 } ' 文件名 '	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

颜色是 rgb24 value, else BLACK / WHITE or ORANGE / BLUE / 绿色 / 青色 / 红 ED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

LOGIC 显示可用于显示以高速捕获的数据。在下面的示例中，P2 通过智能引脚以 333 Mbaud 速率生成 8-N-1 串行数据。流式传输器可以捕获此位流。每经过一个时钟周期，流式传输器都会记录智能引脚的 IN 信号及其从相邻引脚读取的输出状态。每当它获得四个两位样本集时，它执行 RFBYTE 操作以将其保存到枢纽 RAM 中，从而形成连续的字节、单词和 LONGS。通过调用 LONGS_2BIT 打包数据模式，我们可以让 LOGIC 显示从 LONGS 中解包两位样本集，从而产生每个 LONGS 16 个样本。

```
CON_clkfreq 1_333_333      'go really fast, 3ns clock period
rxpin      = 24             奇偶引脚
txpin      _      +1       奇数引脚
样本       = 32             16 倍样本
缓冲长度 样本 / 16         每个长字包含 16 个 2 位样本
x          _               流式传输器模式                e
mode

VAR 缓冲缓冲长度

PUB go() | i, 缓冲区 addr

    调试 ( ` 逻辑 串行样本 `(样本) 间距 12 'TX' 'IN' LONGS_2BIT ) 调试 ( ` 串行触发 十 + 22 )

    buffaddr := @buff

    重复
        org
        wrpin      _ ,#rxpin      rxpin 输入 txpin at rxpin+1

        wrpin      正常           设置 txpin 的异步发送模式
        wxpin      _-1, txpin     将波特率设置为 =sysclock/1, 大小 =8。
        dirh       # txpin        启用智能引脚

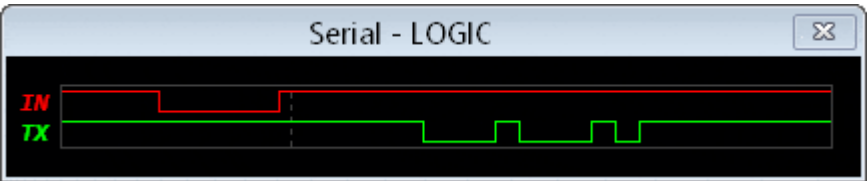
        wrfast     #0, 缓冲区地址 设置缓冲区为快速写模式
        xmode,#0   开始捕获 2-bit 样本

        i,#txpin   传输字节

        等待 wifi           等待流捕获完成
        调试内存利用率

        调试 (v1) 串行 (v2) 十六进制长数组 _(@buff, 缓冲长度 )

        等待 (20)
```



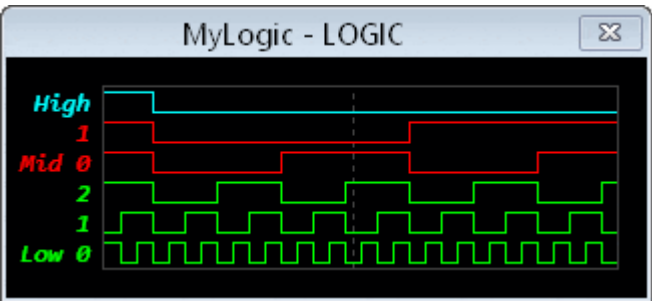
LOGIC Display Logic analyzer with single and multi-bit labels, 1..32 channels, can trigger on pattern

```
CON_clkfreq  = 10_000_000

PUB go() | i

    debug(`LOGIC MyLogic SAMPLES 32 'Low' 3 'Mid' 2 'High')
    debug(`MyLogic TRIGGER $07 $04 HOLDOFF 2)

    repeat
        debug(`MyLogic `(i & 63))
        i++
        waitms(25)
```



LOGIC Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SAMPLES 4_to_2048	Set the number of samples to track and display.	32
SPACING 2_to_32	Set the sample spacing. The width of the display will be SAMPLES * SPACING.	8
RATE 1_to_2048	Set the number of samples (or triggers, if enabled) before each display update.	1
LINESIZE 1_to_7	Set the line size.	1
TEXTSIZE 6_to_200	Set the legend text size. Height of text determines height of logic levels.	editor text size
COLOR back_color {grid_color}	Set the background and grid colors *.	BLACK, GREY 4
'name' {1_to_32 {color}}	Set the first/next channel or group name, optional bit count, optional color *.	1, default color
packed_data_mode	Enable packed-data mode. See description at end of this section.	<none>
LOGIC Feeding	Description	Default
TRIGGER mask match sample_offset	Trigger on (data & mask) = match. If mask = 0, trigger is disabled.	0, 1, SAMPLES / 2
HOLDOFF 2_to_2048	Set the minimum number of samples required from trigger to trigger.	SAMPLES
data	Numerical data is applied LSB-first to the channels.	
CLEAR	Clear the sample buffer and display, wait for new data.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

* Color is rgb24 value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

The LOGIC display can be used to display data that was captured at high speed. In the example below, the P2 is generating 8-N-1 serial at 333 Mbaud using a smart pin. This bit stream can be captured by the streamer. On every clock, the streamer will record the smart pin's IN signal and its output state, as read from an adjacent pin. Every time it gets four two-bit sample sets, it does an RFBYTE to save them to hub RAM, forming contiguous bytes, words, and longs. By invoking the LONGS_2BIT packed-data mode, we can have the LOGIC display unpack the two-bit sample sets from longs, yielding 16 sets per long.

```
CON_clkfreq  = 333_333_333  'go really fast, 3ns clock period
rxpin      = 24             'even pin
txpin      = rxpin+1       'odd pin
samps      = 32             'multiple of 16 samples
bufflongs  = samps / 16     'each long holds 16 2-bit samples
xmode      = $D0800000 + rxpin << 17 + samps  'streamer mode

VAR buff[bufflongs]

PUB go() | i, buffaddr

    debug(`logic Serial samples `(samps) spacing 12 'TX' 'IN' longs_2bit)
    debug(`Serial trigger %10 %10 22)
    buffaddr := @buff

    repeat
        org
        wrpin      ##+1<<28,#rxpin      'rxpin inputs txpin at rxpin+1

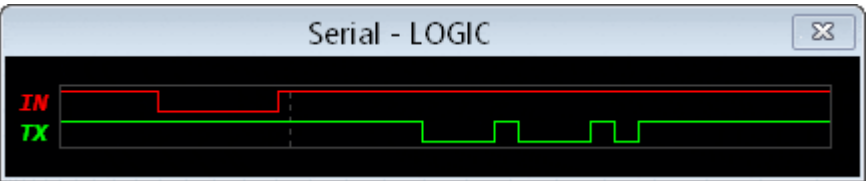
        wrpin      #%01_11110_0,#txpin   'set async tx mode for txpin
        wxpin      ##1<<16+8-1,#txpin     'set baud=sysclock/1 and size=8
        dirh       #txpin                'enable smart pin

        wrfast     #0,buffaddr           'set write-fast at buff
        xinit      ##xmode,#0           'start capturing 2-bit samples

        wypin      i,#txpin              'transmit serial byte

        waitxfi                                'wait for streamer capture done
        end

        debug(`Serial `uhex_long_array_(@buff, bufflongs))
        i++
        waitms(20)
```



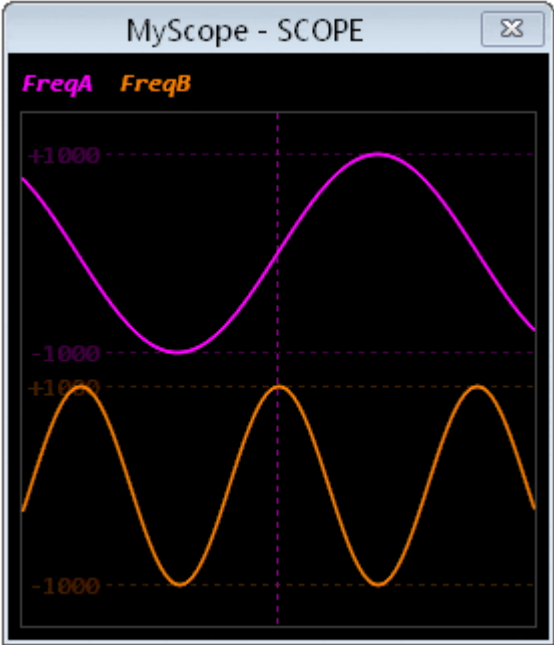
示波器 显示 示波器，具有 1..8 个通道，可以对具有滞后 电平的信号进行触发。

```
CON _clkfreq  = 100_000_000

PUB go() | a, af, b, bf

    debug(`SCOPE MyScope)
    debug(`MyScope 'FreqA' -1000 1000 100 136 15 MAGENTA)
    debug(`MyScope 'FreqB' -1000 1000 100 20 15 ORANGE)
    debug(`MyScope TRIGGER 0 HOLDOFF 2)

    repeat
        a := qsin(1000, af++, 200)
        b := qsin(1000, bf++, 99)
        debug(`MyScope `(a,b))
        waitus(200)
```



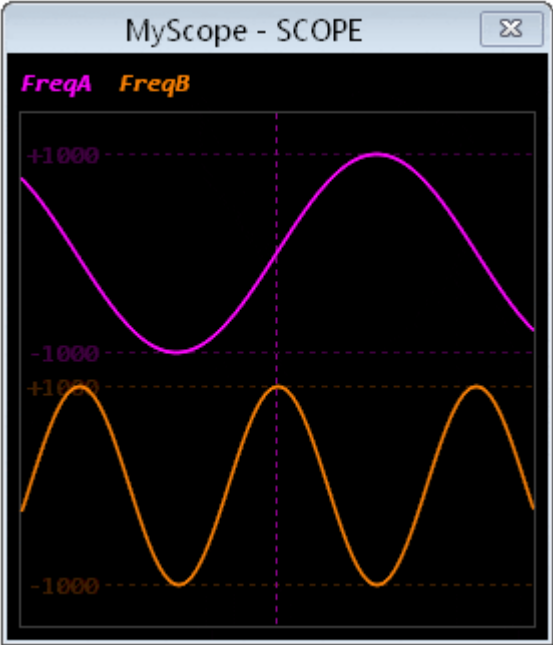
SCOPE Display Oscilloscope with 1..8 channels, can trigger on level with hysteresis

```
CON _clkfreq  = 100_000_000

PUB go() | a, af, b, bf

    debug(`SCOPE MyScope)
    debug(`MyScope 'FreqA' -1000 1000 100 136 15 MAGENTA)
    debug(`MyScope 'FreqB' -1000 1000 100 20 15 ORANGE)
    debug(`MyScope TRIGGER 0 HOLDOFF 2)

    repeat
        a := qsin(1000, af++, 200)
        b := qsin(1000, bf++, 99)
        debug(`MyScope `(a,b))
        waitus(200)
```



示波器 实例化	描述	默认
标题 'string'	将窗口标题设置为“字符串”。	无
POS 左上角	设置窗口位置。	0, 0
大小 宽度高度	设置显示尺寸 (32..2048 x 32..2048)	255, 256
样本 16_to_2048	设置要跟踪和显示的样本数。	256
速率 1_to_2048	在每次显示更新之前，设置样本数（或触发数，如果已启用）。	—
点大小 0_到_32	设置像素大小以显示精确的样本点。	0
线条大小 0_to_32	将连接样本点的线条粗细设置为半像素。	3
文本大小 6_to_200	设置图例文本大小。	编辑器文本大小
颜色 背景颜色 网格颜色	设置背景色和网格颜色。	黑色，灰色 4
打包数据模式	启用打包数据模式。请参阅本节末尾的描述。	<none>
示波器 Feeding	描述	默认
名称 {min {max {y 尺寸 {y 基准 { 图例 { 颜色 }}}}}	设置第一个 / 下一个通道名称、最小值、最大值、y 尺寸、y 基准、图例和颜色 *。图例为 %abcd，其中 %a 到 %d 启用最大图例、最小图例、最大线、最小线。	完整，无图例，默认颜色
触发 通道 {arm_level {trigger_level { 偏移量 }}}	设置触发通道、臂级、触发级别和右偏移量。如果通道 1 禁用。	—, —, 0, 宽度 / 2
休止时间 2 到 2048	设置从触发到触发所需的最小样本数。	样本
数据	数值数据按升序应用于通道。	
清除	清除采样缓冲区和显示，等待新数据。	
保存 { 窗口 } '文件名 '	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

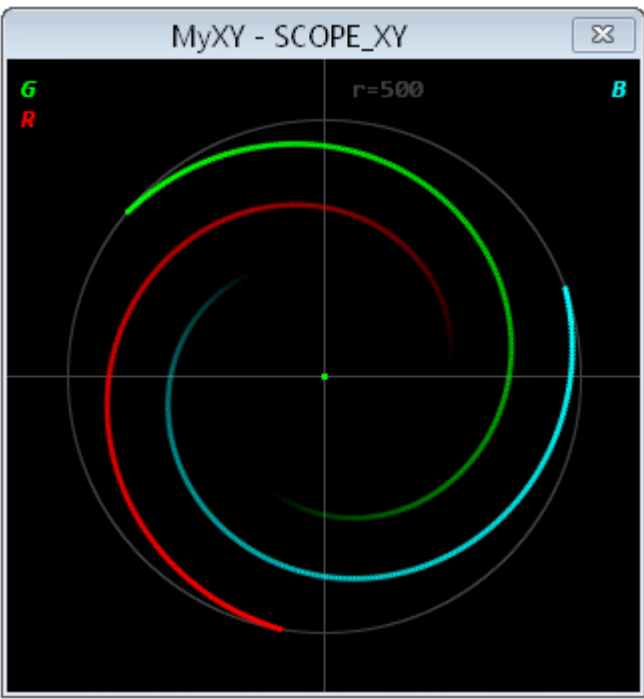
颜色是 rgb24 value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

SCOPE Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SIZE width height	Set the display size (32..2048 x 32..2048)	255, 256
SAMPLES 16_to_2048	Set the number of samples to track and display.	256
RATE 1_to_2048	Set the number of samples (or triggers, if enabled) before each display update.	1
DOTSIZE 0_to_32	Set the dot size in pixels for showing exact sample points.	0
LINESIZE 0_to_32	Set the line size in half-pixels for connecting sample points.	3
TEXTSIZE 6_to_200	Set the legend text size.	editor text size
COLOR back_color {grid_color}	Set the background and grid colors *.	BLACK, GREY 4
packed_data_mode	Enable packed-data mode. See description at end of this section.	<none>
SCOPE Feeding	Description	Default
'name' {min {max {y_size {y_base {legend {color}}}}}}	Set first/next channel name, min value, max value, y size, y base, legend, and color *. Legend is %abcd, where %a to %d enable max legend, min legend, max line, min line.	full, no legend, default color
TRIGGER channel {arm_level {trigger_level {offset}}}	Set the trigger channel, arm level, trigger level, and right offset. If channel=-1, disabled.	-1, -1, 0, width / 2
HOLDOFF 2_to_2048	Set the minimum number of samples required from trigger to trigger.	SAMPLES
data	Numerical data is applied to the channels in ascending order.	
CLEAR	Clear the sample buffer and display, wait for new data.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

* Color is rgb24 value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

SCOPE_XY Display 带有 1..8 个通道的 XY 示波器，持续时间为 1.512 个样本，极坐标模式，对数刻度模式

```
调试 = 100000000 PUB go() | i debug(`SCOPE_XY MyXY 范围
500 极坐标 360 'G' 'R' 'B') 重复 重复 i 从 0 到 500 debug(` MyXY `
(i, i, i, i+120, i, i+240)) 等待 (5)
```

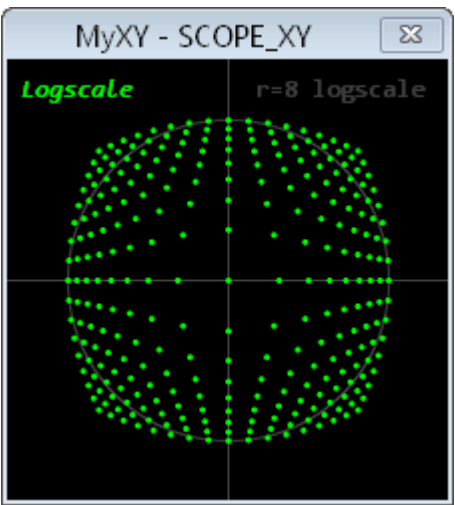
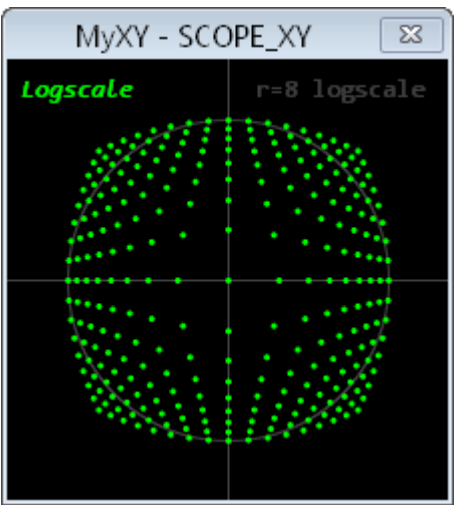


SCOPE_XY 实例化	描述	默认
标题 'string'	将窗口标题设置为“字符串”。	无
POS 左上角	设置窗口位置。	0, 0
大小 半径	设置显示半径为像素。	128
1_to_7FFFFFFF 范围	为接收数据设置单位圆半径	7FFFFFFF
样本 0_to_512	设置要跟踪和显示的样本数量，并设置持续时间。使用 0 表示无限持续时间。	256
速率 1_to_512	在每次显示更新之前设置样本数。	—
点大小 2_to_20	将样本点的点的大小设置为半像素。	调试内存利用率
文本大小 6_to_200	设置图例文本大小。	编辑器文本大小
颜色 背景颜色 网格颜色	设置背景和网格颜色。	黑色，灰色 4
极坐标 {twopi { 偏移量 }}	设置极坐标模式、twopi 值和偏移量。对于 twopi 值为 100000000 或 -100000000 的情况，使用 0 或 -1。	十
对数刻度	将对数刻度模式设置为放大单位圆内的点。	关闭
名称 { 颜色 }	设置第一个 / 下一个通道名称，并可选地为其分配颜色。	默认颜色
打包数据模式	启用打包数据模式。请参阅本节末尾的描述。	无
示波器_XY 馈送	描述	默认
x y	X-Y 数据对按升序应用于通道。在极坐标模式下，x= 长度和 y= 角度。	
清除	清除采样缓冲区和显示，等待新数据。	
保存 { 窗口 } ' 文件名 '	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

颜色是 rgb24 value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

```
CON_clkfreq 1_000_000      正常模式

调试 PUB go() | x, y 示波器
(`SCOPE_XY)                    MyXY SIZE 80 RANGE 8 SAMPLES 0 'Normal')
  重复 x 从 -8 到 8 重复 y 从 -
  8 到 8 调试(`MyXY `(x,y))
```



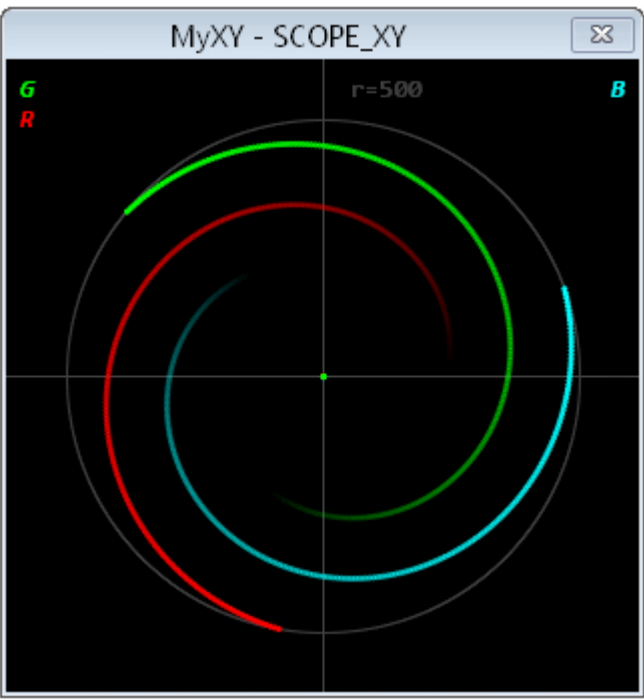
SCOPE_XY Display XY oscilloscope with 1..8 channels, persistence of 1.512 samples, polar mode, log scale mode

```
CON_clkfreq = 100_000_000

PUB go() | i

  debug(`SCOPE_XY MyXY RANGE 500 POLAR 360 'G' 'R' 'B')

  repeat
    repeat i from 0 to 500
      debug(`MyXY `(i, i, i, i+120, i, i+240))
      waitms(5)
```

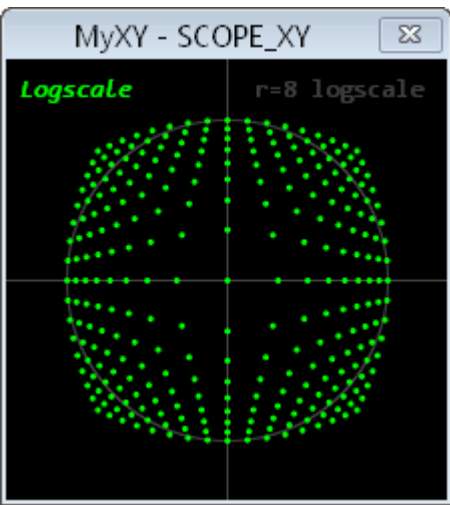
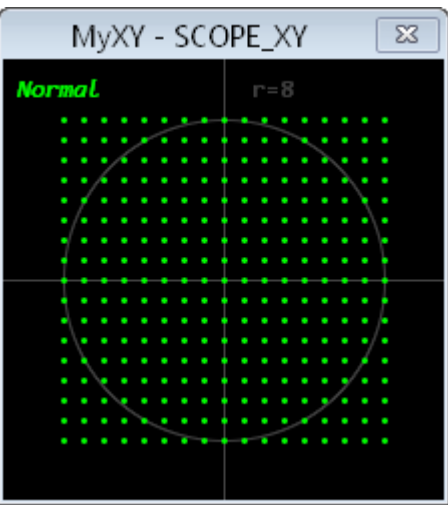


SCOPE_XY Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SIZE radius	Set the display radius in pixels.	128
RANGE 1_to_7FFFFFFF	Set the unit circle radius for incoming data	\$7FFFFFFF
SAMPLES 0_to_512	Set the number of samples to track and display with persistence. Use 0 for infinite persistence.	256
RATE 1_to_512	Set the number of samples before each display update.	1
DOTSIZE 2_to_20	Set the dot size in half-pixels for showing sample points.	6
TEXTSIZE 6_to_200	Set the legend text size.	editor text size
COLOR back_color {grid_color}	Set the background and grid colors *.	BLACK, GREY 4
POLAR {twopi {offset}}	Set polar mode, twopi value, and offset. For a twopi value of \$100000000 or -\$100000000, use 0 or -1.	\$100000000, 0
LOGSCALE	Set log-scale mode to magnify points within the unit circle.	<off>
'name' {color}	Set the first/next channel name and optionally assign it a color *.	default color
packed_data_mode	Enable packed-data mode. See description at end of this section.	<none>
SCOPE_XY Feeding	Description	Default
x y	X-Y data pairs are applied to the channels in ascending order. In polar mode, x=length and y=angle.	
CLEAR	Clear the sample buffer and display, wait for new data.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

* Color is rgb24 value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

```
CON_clkfreq = 10_000_000      'Normal mode

PUB go() | x, y
  debug(`SCOPE_XY MyXY SIZE 80 RANGE 8 SAMPLES 0 'Normal')
  repeat x from -8 to 8
    repeat y from -8 to 8
      debug(`MyXY `(x,y))
```



快速傅里叶变换 包含 1..8 个通道、 4..2048 个点、窗口化结果、对数刻度模式的快速傅里叶变换显示

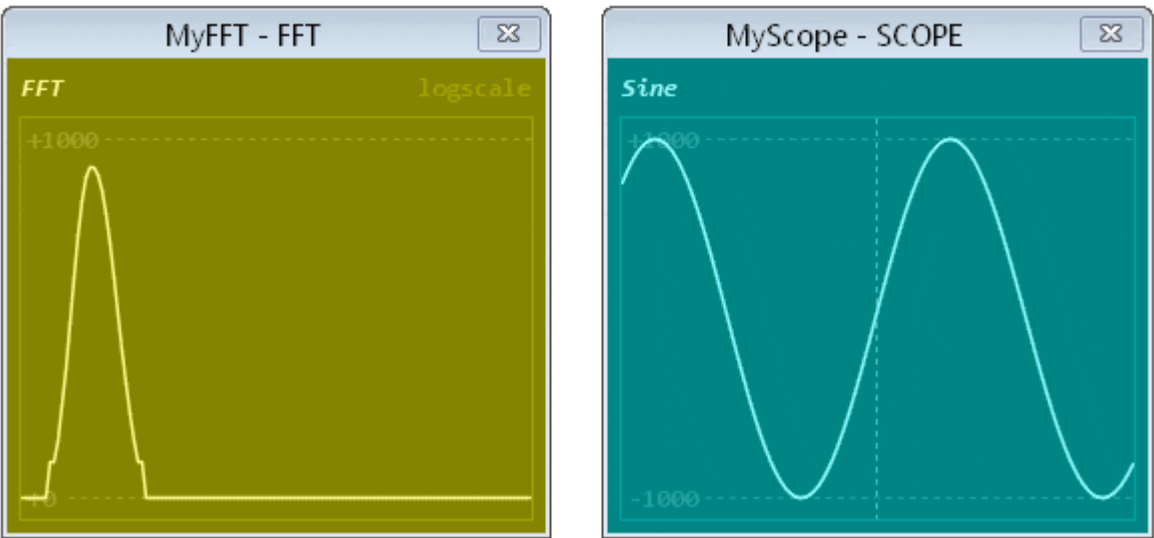
```
CON _clkfreq = 100_000_000

PUB go() | i, j, k

' Set up FFT
debug(`FFT MyFFT SIZE 250 200 SAMPLES 2048 0 127 RATE 256 LOGSCALE COLOR YELLOW 4 YELLOW 5)
debug(`MyFFT 'FFT' 0 1000 180 10 15 YELLOW 12)

' Set up SCOPE
debug(`scope MyScope POS 300 0 SIZE 255 200 COLOR CYAN 4 CYAN 5)
debug(`MyScope 'Sine' -1000 1000 180 10 15 CYAN 12)
debug(`MyScope TRIGGER 0)

repeat
  j += 1550 + qsin(1300, i++, 31_000)
  k := qsin(1000, j, 50_000)
  debug(`MyFFT MyScope `(k))
  waitus(100)
```



快速傅里叶变换 实例化	描述	默认
标题 'string'	将窗口标题设置为 “ 字符串”。	无
POS 左上角	设置窗口位置。	0,0
大小 宽度高度	设置显示尺寸 (32..2048 x 32..2048)	256, 256
样本 4_to_2048 {first {last}}	设置 2n 个 FFT 输入点，加上第一个和最后一个结果值以显示。	512, 0, 255
速率 1_to_2048	在每次显示更新之前设置样本数。	样本
点大小 0_到_32	设置像素大小以显示精确的样本点。	0
线条大小 neg32_to_32	将连接样本点的线条粗细设置为半像素。负线条粗细将产生孤立的垂直线。	3
文本大小 6_to_200	设置图例文本大小。	编辑器文本大小
颜色 背景颜色 网格颜色	设置背景色和网格颜色。	黑色，灰色 4
对数刻度	将对数刻度模式设置为放大低级结果。	关闭
打包数据模式	启用打包数据模式。请参阅本节末尾的描述。	无
FFT 馈送	描述	默认
名称 { 放大倍数 { 最大 {y 尺寸 {y 基准 { 图例 { 颜色 } } } } }	设置第一个 / 下一个通道名称、放大因子（2n，n = 0..11），最大振幅、y 尺寸、y 基准、图例，以及颜色 *。图例是 %abcd，其中 %a 到 %d 启用最大图例、最小图例、最大线、最小线。	完整，无图例，默认颜色
数据	数值数据被输入到通道的滑动汉宁窗中，然后 FFT 计算功率水平。	
清除	清除采样缓冲区和显示，等待新数据。	
保存 { 窗口 } ' 文件名 '	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

颜色是 rgb24 value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

FFT Display Fast Fourier Transform with 1..8 channels, 4..2048 points, windowed results, log scale mode

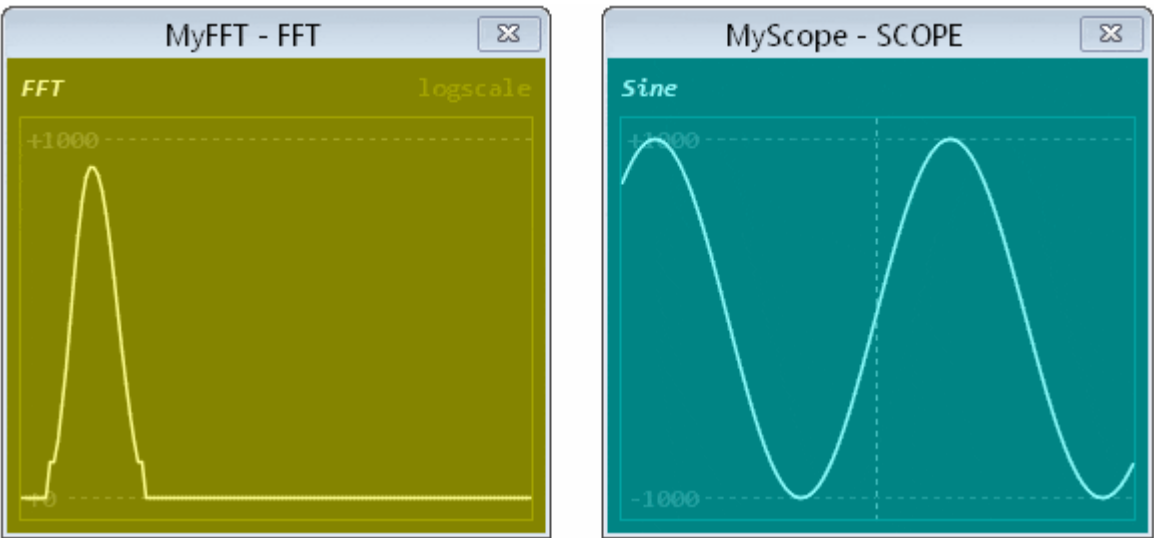
```
CON _clkfreq = 100_000_000

PUB go() | i, j, k

' Set up FFT
debug(`FFT MyFFT SIZE 250 200 SAMPLES 2048 0 127 RATE 256 LOGSCALE COLOR YELLOW 4 YELLOW 5)
debug(`MyFFT 'FFT' 0 1000 180 10 15 YELLOW 12)

' Set up SCOPE
debug(`scope MyScope POS 300 0 SIZE 255 200 COLOR CYAN 4 CYAN 5)
debug(`MyScope 'Sine' -1000 1000 180 10 15 CYAN 12)
debug(`MyScope TRIGGER 0)

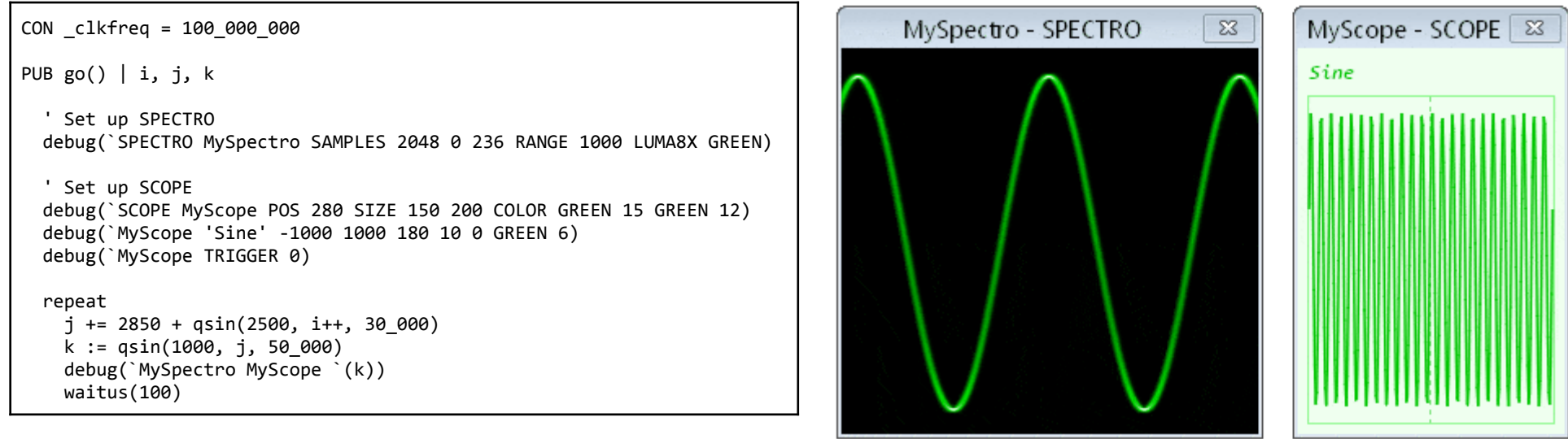
repeat
  j += 1550 + qsin(1300, i++, 31_000)
  k := qsin(1000, j, 50_000)
  debug(`MyFFT MyScope `(k))
  waitus(100)
```



FFT Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SIZE width height	Set the display size (32..2048 x 32..2048)	256, 256
SAMPLES 4_to_2048 {first {last}}	Set the 2 ⁿ number of FFT inputs points, plus the first and last result values to display.	512, 0, 255
RATE 1_to_2048	Set the number of samples before each display update.	SAMPLES
DOTSIZE 0_to_32	Set the dot size in pixels for showing exact sample points.	0
LINESIZE neg32_to_32	Set the line size in half-pixels for connecting sample points. A negative line size will make isolated vertical lines.	3
TEXTSIZE 6_to_200	Set the legend text size.	editor text size
COLOR back_color {grid_color}	Set the background and grid colors *.	BLACK, GREY 4
LOGSCALE	Set log-scale mode to magnify low-level results.	<off>
packed_data_mode	Enable packed-data mode. See description at end of this section.	<none>
FFT Feeding	Description	Default
'name' {mag {max {y_size {y_base {legend {color}}}}}}	Set the first/next channel name, magnification factor (2 ⁿ , n = 0..11), max amplitude, y size, y base, legend, and color *. Legend is %abcd, where %a to %d enable max legend, min legend, max line, min line.	full, no legend, default color
data	Numerical data is fed into the channels' sliding Hanning windows from which the FFT computes power levels.	
CLEAR	Clear the sample buffer and display, wait for new data.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

* Color is rgb24 value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

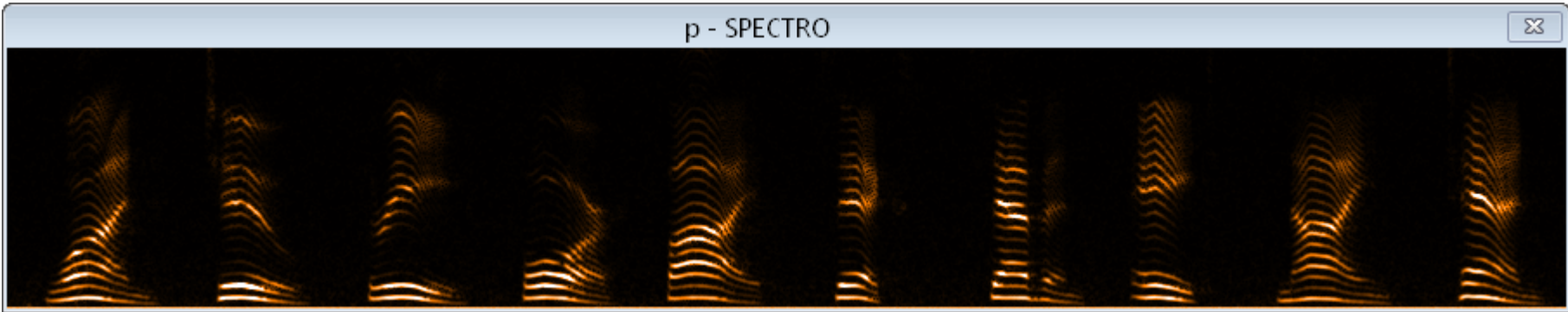
SPECTRO 显示 配备 4 到 2048 点 FFT 、相位着色和对数刻度模式的光谱仪



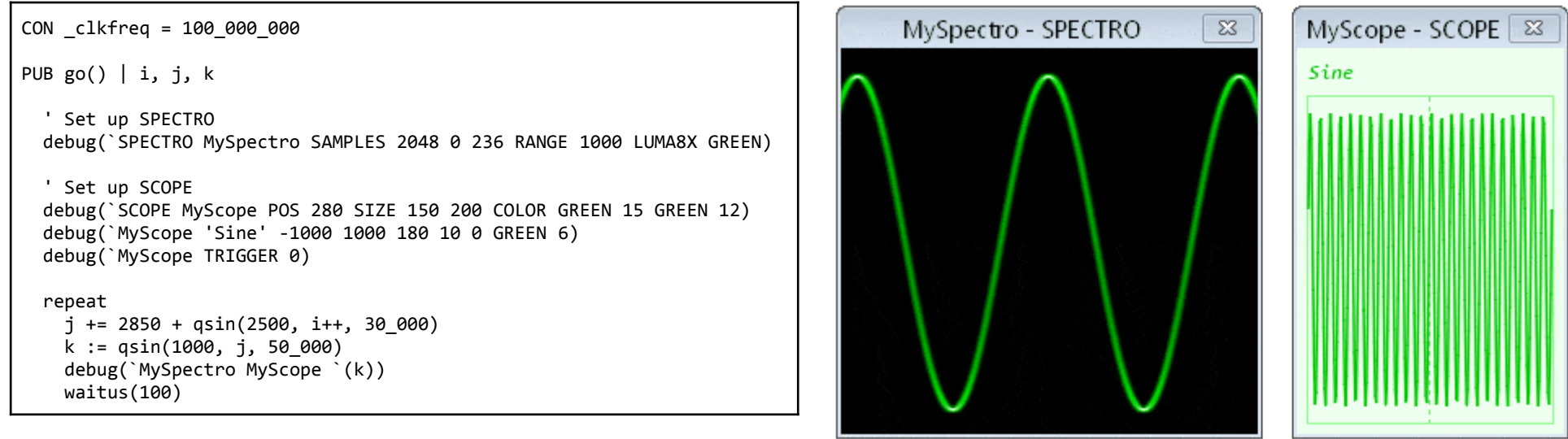
频谱 实例化	描述	默认
标题 'string'	将窗口标题设置为 “ 字符串 ”。	无
POS 左上角	设置窗口位置。	0,0
样本 4_to_2048 {first {last}}	设置 2n 个 FFT 输入点，加上第一和最后一个结果值以显示（定义显示高度）。	512, 0, 255
深度 1_to_2048	设置要显示的垂直线 FFT 结果数量（定义显示宽度）。	256
放大倍数 0_to_11	设置放大因子 (2n, n = 0..11)。	0
范围 饱和和功率	设置像素亮度饱和的功率水平。	7FFFFFFF
速率 1_to_2048	在每次显示更新之前设置样本数。	样本 / 8
追踪 0_to_15	设置追踪模式（请参阅位图显示中的追踪动画）。	15（右，上，滚动）
点大小 宽度和高度 {height}	设置光谱仪的像素宽度和像素高度，或者独立设置它们dently.	1, 1
亮度或 HSV { 颜色或相位 }	将颜色方案设置为 Luma8(W)(X) 颜色，或使用 0..255 范围内的 HSV16(W)(X) 颜色方案相位着色偏移量。	LUMA8X 橙色
对数刻度	将对数刻度模式设置为放大低级结果。	关闭
打包数据模式	启用打包数据模式。请参阅本节末尾的描述。	无
SPECTRO 喂养	描述	默认
数据	数值数据被输入到滑动汉宁窗中，然后进行 FFT 计算功率和相位。	
清除	清除采样缓冲区和显示，等待新数据。	
保存 { 窗口 } ' 文件名 '	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

颜色 i s ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY.

以下，一个 SPECTRO 显示器接收来自麦克风引脚的 ADC 样本。这在频谱上看起来就像口头数数“1”到“10”一样。“1”在左侧，“10”在右侧。水平趋势线之间的垂直距离是声门频率。较大的亮度趋势是语音表音元。这表明了我们的耳朵如何感知语音：



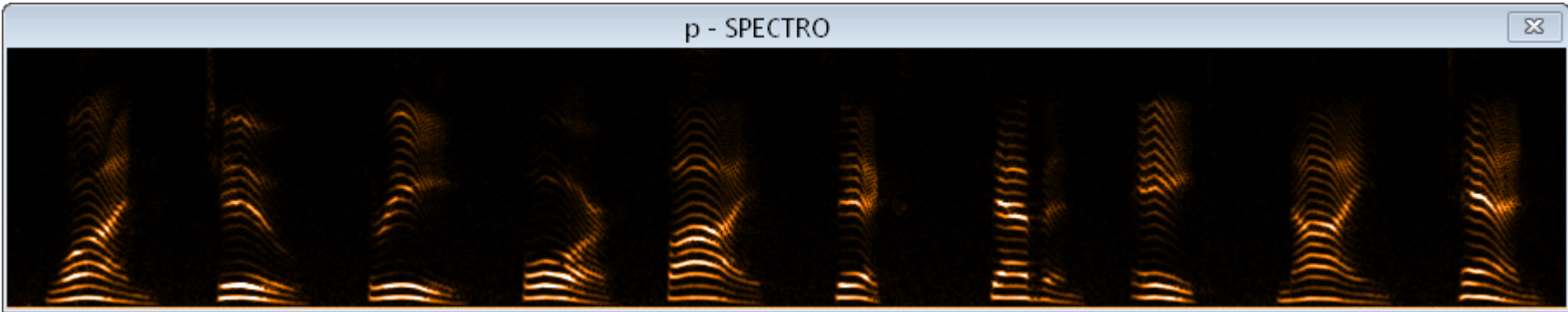
SPECTRO Display Spectrograph with 4..2048-point FFT, phase-coloring, and log scale mode



SPECTRO Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SAMPLES 4_to_2048 {first {last}}	Set the 2 ⁿ number of FFT input points, plus the first and last result values to display (defines display height).	512, 0, 255
DEPTH 1_to_2048	Set the number of vertical-line FFT results to display (defines the display width).	256
MAG 0_to_11	Set the magnification factor (2 ⁿ , n = 0..11).	0
RANGE saturation_power	Set the power level at which pixel brightness saturates.	\$7FFFFFFF
RATE 1_to_2048	Set the number of samples before each display update.	SAMPLES / 8
TRACE 0_to_15	Set the trace pattern (see TRACE animation in BITMAP Display).	15 (right, up, scroll)
DOTSIZE width_and_height {height}	Set the spectrograph pixel-width and pixel-height together, or set them independently.	1, 1
luma_or_hsv {color_or_phase}	Set the color scheme to LUMA8(W)(X) with color *, or HSV16(W)(X) with 0..255 phase-coloring offset.	LUMA8X ORANGE
LOGSCALE	Set log-scale mode to magnify low-level results.	<off>
packed_data_mode	Enable packed-data mode. See description at end of this section.	<none>
SPECTRO Feeding	Description	Default
data	Numerical data is fed into a sliding Hanning window from which the FFT computes power and phase.	
CLEAR	Clear the sample buffer and display, wait for new data.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

* Color is ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY.

Below, a SPECTRO display was fed ADC samples from a pin attached to a microphone. This is what verbally counting from "1" to "10" looks like, spectrally. The "1" is on the left and the "10" is on the right. The vertical distance between horizontal trend lines is glottal pitch. The larger brightness trends are vocal formants. This gives some idea of how our ears perceive speech:



```
CON _clkfreq = 10_000_000

PUB go(): i, j, k

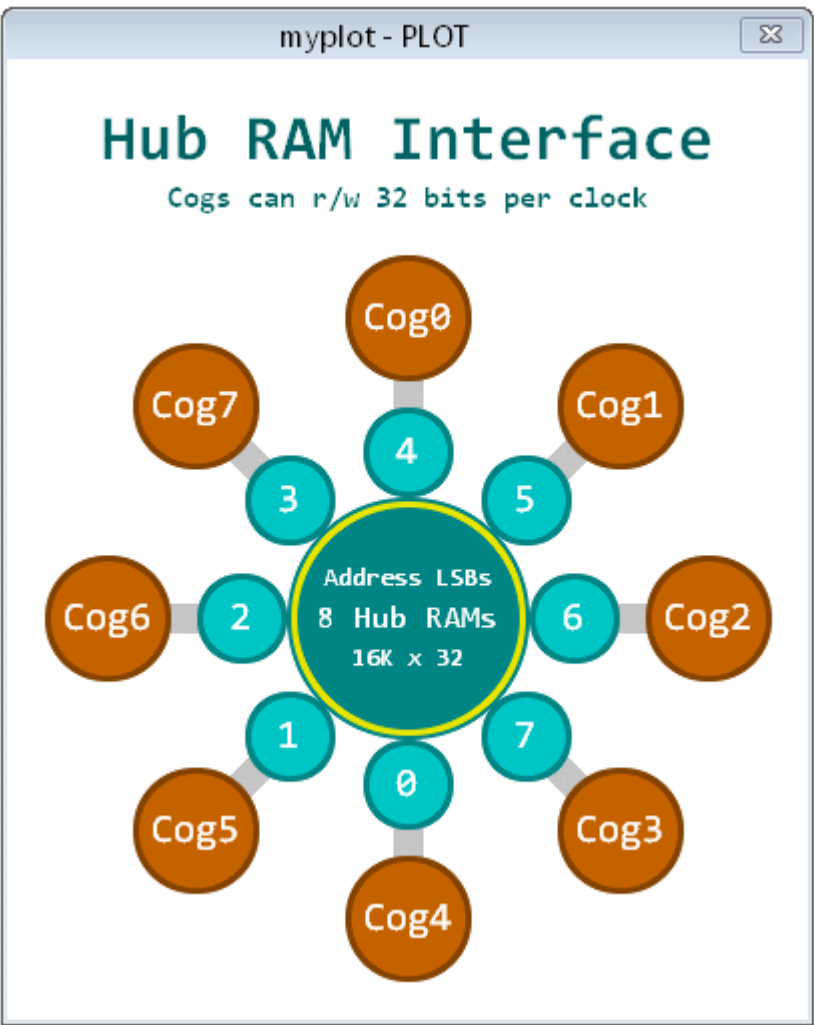
  debug(`plot myplot size 400 480 bgcolor white update)
  debug(`myplot origin 200 200 polar -64 -16)
  k~
  repeat
    debug(`myplot clear)
    debug(`myplot set 240 0 cyan 3 text 24 3 'Hub RAM Interface')
    debug(`myplot set 210 0 text 11 3 'Cogs can r/w 32 bits per clock')

    if k & 8    'move RAMs or draw spokes?
      j++
    else
      repeat i from 0 to 7
        debug(`myplot grey 12 set 83 `(i*8) line 150 `(i*8) 15)

    debug(`myplot set 0 0 cyan 4 circle 121 yellow 7 circle 117 3)
    debug(`myplot set 20 0 white text 10 'Address LSBs')
    debug(`myplot set 0 0 text 12 1 '8 Hub RAMs')
    debug(`myplot set 20 32 text 10 '16K x 32' )

    repeat i from 0 to 7    'draw RAMs and cogs
      debug(`myplot cyan 6 set 83 `(i*8-j) circle 43 text 14 ``(i'))
      debug(`myplot cyan 4 set 83 `(i*8-j) circle 45 3)
      debug(`myplot orange 6 set 150 `(i*8) circle 61 text 13 'Cog'(i)')
      debug(`myplot orange 4 set 150 `(i*8) circle 63 3)

  debug(`myplot update `dly(30))
  k++
```



```
CON _clkfreq = 10_000_000

PUB go(): i, j, k

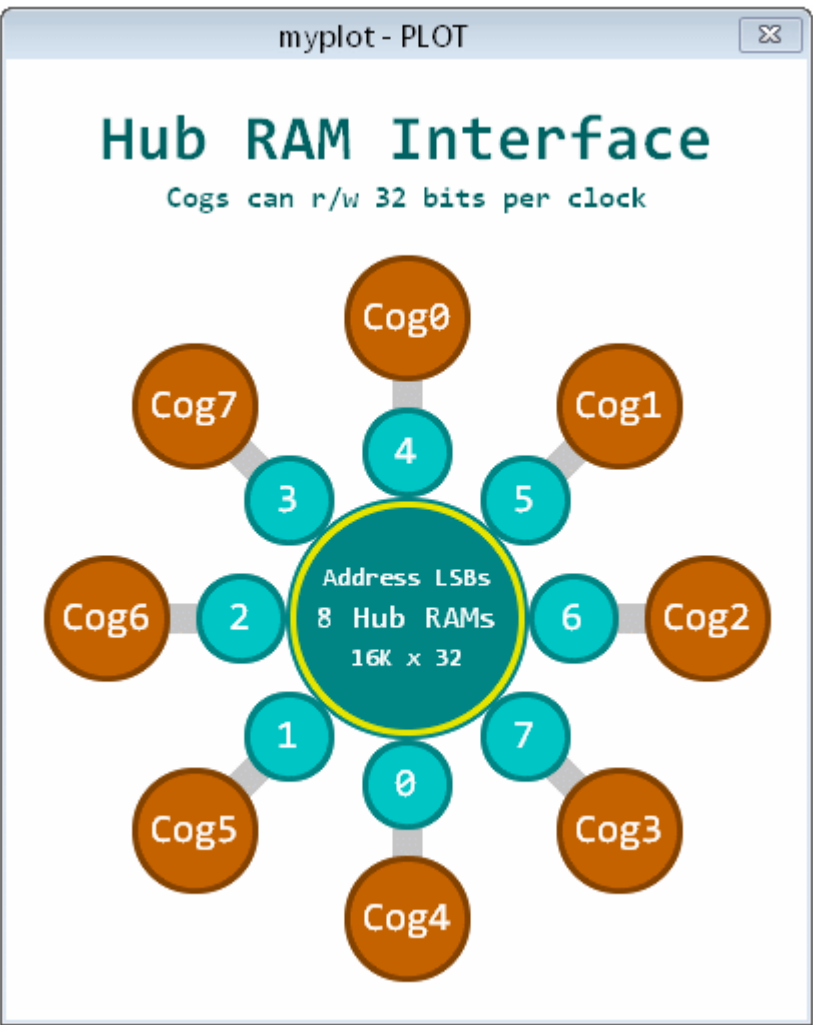
  debug(`plot myplot size 400 480 bgcolor white update)
  debug(`myplot origin 200 200 polar -64 -16)
  k~
  repeat
    debug(`myplot clear)
    debug(`myplot set 240 0 cyan 3 text 24 3 'Hub RAM Interface')
    debug(`myplot set 210 0 text 11 3 'Cogs can r/w 32 bits per clock')

    if k & 8    'move RAMs or draw spokes?
      j++
    else
      repeat i from 0 to 7
        debug(`myplot grey 12 set 83 `(i*8) line 150 `(i*8) 15)

    debug(`myplot set 0 0 cyan 4 circle 121 yellow 7 circle 117 3)
    debug(`myplot set 20 0 white text 10 'Address LSBs')
    debug(`myplot set 0 0 text 12 1 '8 Hub RAMs')
    debug(`myplot set 20 32 text 10 '16K x 32' )

    repeat i from 0 to 7    'draw RAMs and cogs
      debug(`myplot cyan 6 set 83 `(i*8-j) circle 43 text 14 ``(i'))
      debug(`myplot cyan 4 set 83 `(i*8-j) circle 45 3)
      debug(`myplot orange 6 set 150 `(i*8) circle 61 text 13 'Cog'(i)')
      debug(`myplot orange 4 set 150 `(i*8) circle 63 3)

  debug(`myplot update `dly(30))
  k++
```



绘图 实例化	描述	默认
标题 'string'	将窗口标题设置为“字符串”。	<none>
POS 左上角	设置窗口位置。	0,0
大小 宽度高度	设置显示宽度（32..2048）和高度（32..2048）。	256, 256
点大小 宽度和高度 {height}	Set the display pixel-width and pixel-height together, or set them independently.	—, 1
lut1_to_rgb24	设置颜色模式。	RGB24
LUTCOLORS rgb24 rgb24 ...	对于 LUT1..LUT8 颜色模式，请使用 RGB24 颜色加载 LUT。使用 HEX_LONG_ARRAY_ 加载颜色。	默认颜色 0..7
背景颜色	根据当前颜色模式设置背景颜色。	黑色
更新	设置更新模式。仅在接收到“更新”命令时，显示才会更新。	自动更新
图层馈送	描述	默认
lut1_to_rgb24	设置颜色模式。	RGB24
LUTCOLORS rgb24 rgb24 ...	对于 LUT1..LUT8 颜色模式，请使用 RGB24 颜色加载 LUT，并使用 HEX_LONG_ARRAY_ 加载值。	默认颜色 0..7
背景颜色	根据当前颜色模式设置背景颜色。	黑色
颜色 color	根据当前颜色模式设置绘图颜色。在 TEXT 之前使用以更改文本颜色。*	青色
黑白或橙色 / 蓝色 / 绿色 / 青色 / 红色 / 品红色 / 黄色 / 灰色 { 亮度 }	设置橙色到灰色颜色（默认值为 8）的绘制颜色和可选的 0..15 亮度。	青色
透明度 电平	设置 DOT、LINE、CIRCLE、OVAL、BOX 和 OBOX 绘制的透明度级别。0.255 = 清除..透明。	255
精确	切换精确模式，其中线条粗细以及 DOT 和 LINE 的 (x,y) 坐标以像素的 256 分之几表示。	禁用
线条大小	为 DOT 和 LINE 绘制设置线条粗细（像素）。	—
原点 {x_pos y_pos}	将原点设置为笛卡尔坐标系 (x_pos, y_pos)，如果没有指定值，则设置为当前坐标系 (x, y)。	0,0
x y	将绘图位置设置为 (x, y)。LINE 之后，终点将成为新的绘图位置。	
点 { 线条大小 { 透明度 } }	在当前位置绘制一个带有可选线条大小和透明度覆盖的点。	
线 x y { 线条大小 { 透明度 } }	从当前位置绘制一条线到 (x,y)，可选择覆盖线条大小和透明度。	
圆直径 { 线条大小 { 透明度 } }	在可选线条粗细（无 /0 = 实线）和透明度覆盖的情况下，在当前位置绘制一个圆。	
椭圆宽度高度 { 线条大小 { 透明度 } }	在当前位置绘制一个带可选线条粗细（无 /0 = 实线）和透明度覆盖的椭圆。	
框 宽度高度 { 线条大小 { 透明度 } }	在当前位置绘制一个可选线条粗细（无 /0 = 实线）和透明度覆盖的框。	
圆角框 宽度高度 x_ 半径 y_ 半径 线条大小 { 透明度 }	在当前位置绘制一个圆角框，框的宽度、高度、x 和 y 半径以及可选的线条粗细。（none/0 = solid）和 透明度 覆盖。	
文本大小	设置文本大小 (6..200)。	十
字体样式 style_YYXXUIWW	设置文本样式为 %YYXXUIWW：%YY 垂直对齐：%00 = 居中，%10 = 底部，%11 = 顶部。	居中

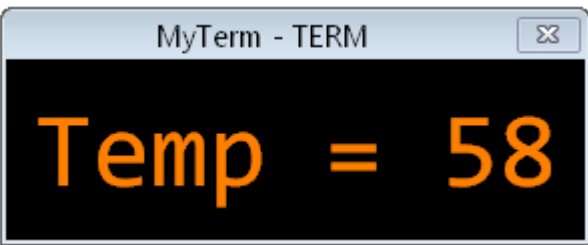
PLOT Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SIZE width height	Set the display width (32..2048) and height (32..2048).	256, 256
DOTSIZE width_and_height {height}	Set the display pixel-width and pixel-height together, or set them independently.	1, 1
lut1_to_rgb24	Set the color mode.	RGB24
LUTCOLORS rgb24 rgb24 ...	For LUT1..LUT8 color modes, load the LUT with rgb24 colors. Use HEX_LONG_ARRAY_ to load colors.	default colors 0..7
BACKCOLOR color	Set the background color according to the current color mode. *	BLACK
UPDATE	Set UPDATE mode. The display will only be updated when fed an 'UPDATE' command.	automatic update
PLOT Feeding	Description	Default
lut1_to_rgb24	Set color mode.	rgb24
LUTCOLORS rgb24 rgb24 ...	For LUT1..LUT8 color modes, load the LUT with rgb24 colors. Use HEX_LONG_ARRAY_ to load values.	default colors 0..7
BACKCOLOR color	Set the background color according to the current color mode. *	BLACK
COLOR color	Set the drawing color according to the current color mode. Use just before TEXT to change text color. *	CYAN
BLACK/WHITE or ORANGE/BLUE/GREEN/CYAN/ RED/MAGENTA/YELLOW/GREY {brightness}	Set the drawing color and optional 0..15 brightness for ORANGE..GREY colors (default is 8).	CYAN
OPACITY level	Set the opacity level for DOT, LINE, CIRCLE, OVAL, BOX, and OBOX drawing. 0..255 = clear..opaque.	255
PRECISE	Toggle precise mode, where line size and (x,y) for DOT and LINE are expressed in 256ths of a pixel.	disabled
LINESIZE size	Set the line size in pixels for DOT and LINE drawing.	1
ORIGIN {x_pos y_pos}	Set the origin point to cartesian (x_pos, y_pos) or to the current (x, y) if no values are specified.	0, 0
SET x y	Set the drawing position to (x, y). After LINE, the endpoint becomes the new drawing position.	
DOT {linesize {opacity}}	Draw a dot at the current position with optional LINESIZE and OPACITY overrides.	
LINE x y {linesize {opacity}}	Draw a line from the current position to (x,y) with optional LINESIZE and OPACITY overrides.	
CIRCLE diameter {linesize {opacity}}	Draw a circle around the current position with optional line size (none/0 = solid) and OPACITY override.	
OVAL width height {linesize {opacity}}	Draw an oval around the current position with optional line size (none/0 = solid) and OPACITY override.	
BOX width height {linesize {opacity}}	Draw a box around the current position with optional line size (none/0 = solid) and OPACITY override..	
OBOX width height x_radius y_radius {linesize {opacity}}	Draw a rounded box around the current position with width, height, x and y radii, and optional line size (none/0 = solid) and OPACITY override.	
TEXTSIZE size	Set the text size (6..200).	10
TEXTSTYLE style_YYXXUIWW	Set the text style to %YYXXUIWW: %YY is vertical justification: %00 = middle, %10 = bottom, %11 = top.	%00000001

	水平对齐：居中，右对齐，左对齐。 下划线： 1 = 下划线。 * 是斜体： %1 = 斜体。 %WW 是粗细： %00 = 细， %01 = 正常， %10 = 粗， 以及 %11 = 粗重。	
文本角度 角度	设置文本角度。在笛卡尔模式下，角度以度为单位。	0
文本 { 大小 { 样式 { 角度 }} } 'text'	绘制带有大小、样式和角度覆盖的文本。要更改文本颜色，请在 TEXT 之前声明一个颜色。	
极坐标 {twopi { 偏移量 }}	设置极坐标模式、twopi 值和偏移量。例如，POLAR -12 -3 类似于钟表。 对于两个 pi 值 100000000 或 -100000000，使用 0 或 -1。 在极坐标模式下，(x, y) 坐标被解释为 (长度 , 角度)。	十
笛卡尔	启用笛卡尔模式。这是默认模式。	
清除	清除绘图，恢复背景颜色。	
更新	更新窗口，显示当前绘图。在更新模式下使用。	
保存 { 窗口 } '文件名 '	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

颜色是 a modal value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

TERM Display 终端显示文本

```
调试 = 10000000 CON go() | i 调试 (^ 终端
MyTerm 大小 9 1 文本大小 40) 重复 调试 (^
MyTerm 1 'Temp = `(i)' 等待 (500)
```



终端 实例化	描述	默认
标题 'string'	将窗口标题设置为“字符串”。	无
POS 左上角	设置窗口位置。	0,0
大小 列 行	设置终端列 (1..256) 和终端行 (1..256) 的数量。	4, 20
文本大小	设置终端文本大小 (6..200)。	编辑器文本大小
颜色 文本颜色 背景颜色 ...	设置文本颜色和背景颜色组合 #0. #3. *	默认颜色
背景颜色	设置显示背景颜色。	黑色
更新	设置更新模式。仅在接收到“更新”命令时，显示才会更新。	自动更新
终端输入	描述	默认
字符	清除终端显示和主屏幕光标。 主页光标。 将列设置为下一个字符值。 将行设置为下一个字符值。 选择颜色组合 #0。 选择颜色组合 #1。 选择颜色组合 #2。 选择颜色组合 #3。 退格。 跳转到下一个第 8 列。 对于 13 或 10 = 换行。 32. 一 . 可打印字符。	
字符串	打印字符串。	
清除	清除显示为背景颜色。	
更新	更新窗口，显示当前文本屏幕。在更新模式下使用。	
保存 { 窗口 } '文件名 '	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

颜色是 a modal value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

	%XX is horizontal justification: %00 = middle, %10 = right, %11 = left. %U is underline: %1 = underline. %I is italic: %1 = italic. %WW is weight: %00 = light, %01 = normal, %10 = bold, and %11 = heavy.	
TEXTANGLE angle	Set the text angle. In cartesian mode, the angle is in degrees.	0
TEXT {size {style {angle}}} 'text'	Draw text with overrides for size, style, and angle. To change text color, declare a color just before TEXT.	
POLAR {twopi {offset}}	Set polar mode, twopi value, and offset. For example, POLAR -12 -3 would be like a clock face. For a twopi value of \$100000000 or -\$100000000, use 0 or -1. In polar mode, (x, y) coordinates are interpreted as (length, angle).	\$100000000, 0
CARTESIAN	Set cartesian mode. This is the default mode.	
CLEAR	Clear the plot to the background color.	
UPDATE	Update the window with the current plot. Used in UPDATE mode.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

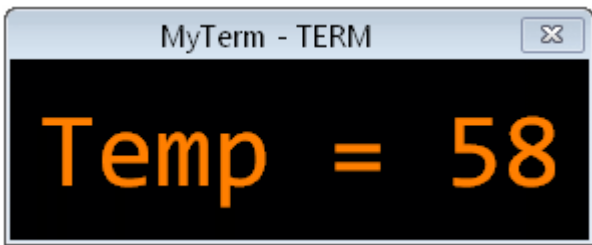
* Color is a modal value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

TERM Display Terminal for displaying text

```
CON _clkfreq = 10_000_000

PUB go() | i

    debug(^ TERM MyTerm SIZE 9 1 TEXTSIZE 40)
    repeat
        repeat i from 50 to 60
            debug(^ MyTerm 1 'Temp = `(i)')
            waitms(500)
```



TERM Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SIZE columns rows	Set the number of terminal columns (1..256) and terminal rows (1..256).	40, 20
TEXTSIZE size	Set the terminal text size (6..200).	editor text size
COLOR text_color back_color ...	Set text-color and background-color combos #0..#3. *	default colors
BACKCOLOR color	Set the display background color. *	BLACK
UPDATE	Set UPDATE mode. The display will only be updated when fed an 'UPDATE' command.	automatic update
TERM Feeding	Description	Default
character	0 = Clear terminal display and home cursor. 1 = Home cursor. 2 = Set column to next character value. 3 = Set row to next character value. 4 = Select color combo #0. 5 = Select color combo #1. 6 = Select color combo #2. 7 = Select color combo #3. 8 = Backspace. 9 = Tab to next 8th column. 13+10 or 13 or 10 = New line. 32..255 = Printable character.	
'string'	Print string.	
CLEAR	Clear the display to the background color.	
UPDATE	Update the window with the current text screen. Used in UPDATE mode.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

* Color is a modal value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).